

Phase 2 Report

Kubernetes Workload Analyzer — demo workloads, verified against live Prometheus

Phase status	DONE (2026-09-25)
Report date	2026-09-25
Repository	github.com/ISHANT57/kubernetes-workload-analyzer
State of main	6570169 (PR #5 merged)
Cluster	kind workload-analyzer , namespace demo , 9 workloads
Written for	the project owner, as a study and reference document

Evidence labels used throughout. **MEASURED** a command was run on this machine and the number is its output · **VERIFIED** checked against upstream documentation, source or a registry · **ESTIMATED** a projection, not measured · **UNVERIFIED** not yet tested; a test is named.

Contents

1. What Phase 2 was for
2. Design decision: agnhost only, not stressing/busybox
3. The duty-cycle technique
4. The nine fixtures, one by one
5. demo/expected-findings.yaml
6. Verification method
7. Live verification results
8. Re-verification 4.5 hours later
9. What Phase 2 confirms for the rule engine (R001-R005)
10. Resource footprint
11. Git and GitHub workflow
12. Current state, operating and rollback
13. Open items and next phase
14. Appendix A — Manifest reference
15. Appendix B — Verification query reference

1. What Phase 2 was for

Phase 1 built and verified the data layer (a kind cluster, a trimmed Prometheus stack, every metric the planned rules depend on). But an idle lab cluster has nothing interesting to find — no over-provisioning, no crashes, no OOMs. Phase 2's job was to build a **deterministic set of broken/inefficient workloads** with known, predictable behaviour, so that:

- Phase 4's rule engine (R001–R005) has real Prometheus data to run against, not synthetic or imagined numbers;
- each rule's correctness can be checked against a written-down expected answer (`demo/expected-findings.yaml`), rather than eyeballed;
- the tricky edge cases identified in Phase 1 — HPA coupling, bursty workloads, insufficient history — have a concrete fixture to exercise them, not just a paragraph in a design doc.

Phase 2 wrote no analyzer code. It is entirely Kubernetes manifests plus verification.

Done-criterion (PHASES.md)

"Deterministic workloads in `demo/`, each mapped to an expected result in `demo/expected-findings.yaml`. Each workload shows its intended behaviour in Prometheus." — **met**, verified twice (once immediately after deployment, once 4.5 hours later — see §8).

2. Design decision: agnhost only, not stress-ng/busybox

The original design (docs/requirements.md, written in Phase 0) specified `ghcr.io/colinianking/stress-ng` for CPU/memory load and `docker.io/library/busybox:1.37` for the crash-loop scenario. Neither was ever actually pulled or tested — they were only checked for *existence* in a registry.

Phase 1's labs, by contrast, proved that `registry.k8s.io/e2e-test-images/agnhost:2.53` and `resource-consumer:1.15.0` work exactly as needed, and — because both images were already used in Phase 1 — **both were already cached on the kind node**. Phase 2 was built entirely on these two images. Result: **zero new image pulls** for the whole demo namespace.

Need	Original plan	What was actually used	Why switched
CPU/memory load	stress-ng (never tested)	agnhost stress subcommand (proven in Phase 1 Lab 03: exact OOM recipe)	Already verified behaviour; already cached
Fractional/duty-cycled CPU	stress-ng --cpu-load (approximate under cgroup limits, per the original design notes)	Hand-rolled shell duty cycle using agnhost's built-in busybox tools (timeout , yes)	Precise and throttle-free — see §3
Crash loop	busybox exit 1	agnhost /bin/sh -c "exit 1"	agnhost's shell was confirmed present in Phase 1 Lab 02 (Alpine-based, has sh/bash/curl/wget/busybox)
HPA-coupled fixture	copy of the over-requested fixture + HPA	resource-consumer + Service + HPA (same shape proven end-to-end in Phase 1 Lab 05)	Reused a mechanism already proven to actually trigger a real scale event

Why this matters. Every mechanism in Phase 2's fixtures had already been individually proven correct in Phase 1's labs before being reused here. Nothing in Phase 2 depended on an untested assumption about how an image behaves under Kubernetes.

3. The duty-cycle technique

Several fixtures need a *specific, known, non-throttled* fraction of a CPU core — not "as much as possible" and not "capped by a limit" (a limit would trigger real cgroup throttling, which R001's over-provisioned rule explicitly excludes via its "throttled ratio < 5%" guard — see docs/requirements.md §3). agnhost's `--cpus` flag only supports whole CPUs, so it cannot produce e.g. "5% of one core" directly.

Instead, each CPU-load fixture runs a small shell loop that alternates between a real, full-intensity busy period and an idle sleep:

```
while true; do timeout <busy-seconds> yes >/dev/null; sleep <idle-seconds>; done
```

`yes` is a standard, well-understood single-core CPU burner — it writes continuously in one process with negligible per-iteration overhead, so during its window it saturates close to 100% of one logical core. `timeout` then kills it after the busy window, and `sleep` holds idle for the rest of the period. The **busy-seconds ÷ (busy+idle) seconds** ratio is the target fraction of one core, in millicores relative to 1000m.

Fixture	Pattern	Target duty	Target usage
right-sized	busy 4s / idle 6s	40%	~400m of a 500m request
cpu-over-requested	busy 1s / idle 19s	5%	~50m of a 1000m request
cpu-spike	busy 30s / idle 270s	10% overall, but 100% during the burst	0m idle → ~1000m burst

Verification of the technique itself (before trusting any fixture built on it): `timeout` and `yes` were confirmed present inside the agnhost image by running `command -v timeout` / `command -v yes` in a live pod before any manifest was written. `busybox date` was checked too and found to lack nanosecond resolution (`%N` unsupported) — ruled out as a timing mechanism, which is why `timeout + yes` was chosen over a hand-timed busy loop.

Memory-only fixtures use agnhost's own `stress --mem-total=<X> --mem-alloc-size=1Mi --mem-alloc-sleep=<N>` directly (no shell wrapper needed) — this is the exact recipe already proven in Phase 1 Lab 03's OOM test, just tuned to different totals. Fixtures needing *both* CPU duty-cycling and a memory hold background the agnhost stress process (`&`) alongside the foreground CPU loop, sharing one container's cgroup.

4. The nine fixtures, one by one

Namespace `demo`, applied as one unit: `kubectl apply -k demo/`. Every object carries the label `k8sa.dev/scenario=<name>`.

4.1 right-sized — negative control

Purpose: prove R001/R002 do *not* fire on a workload whose usage genuinely tracks its request. Without this, a rule that always fires would look correct against the other eight fixtures.

```
args:
- |
  /agnhost stress --cpus=0 --mem-total=60Mi --mem-alloc-size=1Mi --mem-alloc-sleep=2s &
  while true; do timeout 4 yes >/dev/null; sleep 6; done
resources:
  requests: {cpu: 500m, memory: 128Mi}
  limits: {memory: 256Mi}
```

Result: CPU settled 320–460m (64–92% of the 500m request); memory steady ~65–70Mi (51–55% of the 128Mi request). Both stay above R001/R002's "<50% of request" threshold, so neither rule should fire.

4.2 cpu-over-requested

Purpose: a clean R001 signal — real, sustained, low usage against a much larger request, with no CPU limit (so no throttling to trigger R001's guard).

```
args: ["while true; do timeout 1 yes >/dev/null; sleep 19; done"]
resources: {requests: {cpu: 1000m, memory: 64Mi}, limits: {memory: 128Mi}}
```

Result: `rate(container_cpu_usage_seconds_total[2m])` $\approx$ 55–83m — 5.5–8.3% of the 1000m request, well under the 50% threshold.

4.3 memory-over-requested

```
args: ["stress", "--cpus=0", "--mem-total=50Mi", "--mem-alloc-size=1Mi", "--mem-alloc-sleep=5ms"]
resources: {requests: {cpu: 100m, memory: 1Gi}, limits: {memory: 1Gi}}
```

Result: working set steady at ~57–60Mi against a 1Gi request (~5.7%). Limit kept equal to the request (generous, 1Gi) specifically so this fixture can *never* OOM — it must stay a pure resource finding (R002), never accidentally trip a health finding (R004).

4.4 cpu-spike — the bursty guard

Purpose: a workload that is idle 90% of the time but genuinely saturates a full core for 30 seconds every 5 minutes. Its long-run average looks like classic over-provisioning, but a naive rule would be wrong to say so with high confidence — the docs/requirements.md "bursty" exception (`p99/p50 > 4` → cap confidence at LOW) exists specifically for this shape.

```
args: ["while true; do timeout 30 yes >/dev/null; sleep 270; done"]
resources: {requests: {cpu: 500m, memory: 64Mi}, limits: {memory: 128Mi}}
```

Result: polled every 15s across a full 5-minute cycle — 0m for six consecutive samples (90s), then **319m**, then **999m** two samples later, confirming a real full-core burst was caught live, not just inferred from the manifest.

4.5 oom — real out-of-memory kill

```
args: ["stress", "--mem-total=200Mi", "--mem-alloc-size=1Mi", "--mem-alloc-sleep=10ms"]
resources: {requests: {cpu: 50m, memory: 128Mi}, limits: {memory: 128Mi}}
```

Result: `lastState.terminated = reason: OOMKilled, exitCode: 137`, 5 restarts at first check, climbing to 36 over 4.5 hours as kubelet backoff widened toward its 5-minute cap. Exact recipe reused from Phase 1 Lab 03, where this same request/limit/allocation combination was first proven to reliably OOM.

4.6 crashloop

```
command: ["/bin/sh", "-c", "exit 1"]
resources: {requests: {cpu: 50m, memory: 32Mi}, limits: {memory: 64Mi}}
```

Result: `reason: Error, exitCode: 1, pod status CrashLoopBackOff`. Deliberately distinct from the OOM fixture's termination reason — this is what proves R004 (OOM) must key off `reason=OOMKilled` specifically and not fire on every crash loop.

4.7 hpa-coupled

```
image: registry.k8s.io/e2e-test-images/resource-consumer:1.15.0
resources: {requests: {cpu: 400m, memory: 64Mi}, limits: {cpu: "1", memory: 128Mi}}
---
# Service + HorizontalPodAutoscaler (target: cpu Utilization 70%, min 1, max 5)
```

Deliberately no load driver. Unlike Phase 1 Lab 05 (which drove real load to prove the HPA actually rescales), this fixture only needs the HPA *object* to exist, sitting idle. The behaviour under test in Phase 4 is the rule engine's ability to *detect* the HPA and switch to the "HPA-coupled" caveat instead of a plain resize — the actual scaling mechanics were already proven separately, so re-driving load here would only add cost without adding new proof.

Result: idle at ~0m CPU / ~5.4Mi memory against a 400m/64Mi request (which, read naively, would be the strongest over-provisioning signal of any fixture). `kube_horizontalpodautoscaler_info` confirmed present, target metric `cpu Utilization 70`.

4.8 new-workload — the R005 data-quality gate

```
args: ["pause"]
resources: {requests: {cpu: 100m, memory: 32Mi}, limits: {memory: 64Mi}}
```

Purpose: prove R005 (insufficient data) suppresses R001/R002 rather than letting them guess from a too-short window. Deployed at the same time as the other eight — its "newness" needs no special timing, since any rule requiring more history than the fixture's actual age must gate to `insufficient` regardless of when it was created.

Result: idle (~6.5Mi memory, negligible CPU), age well under any of R001/R002's minimum-data windows at check time.

4.9 pending — unschedulable

```
args: ["pause"]
resources: {requests: {cpu: "64"}}
```

Result: pod stuck `Pending; FailedScheduling: 0/1 nodes are available: 1 Insufficient cpu`. No MVP rule consumes this yet — kept for the backlog scheduling-health rule and for testing that the dashboard handles a workload with zero usage data at all.

5. demo/expected-findings.yaml

A single YAML file records the ground truth for all nine fixtures — the mechanism, the actual measured behaviour, and which rules must (or must not) fire. This is what the Phase 4 integration test will diff the analyzer's real output against.

```
scenarios:
- name: cpu-over-requested
  workload: {kind: Deployment, name: cpu-over-requested}
  request: {cpu: 1000m, memory: 64Mi}
  observed: {cpu: "26-83m (avg ~55m via rate(...[2m]))", memory: "~0.6Mi"}
  mechanism: "5% CPU duty cycle (busy 1s / idle 19s), no CPU limit (throttled ratio ~0%)"
  expect:
    rules_fired: [R001]
    confidence: MEDIUM
    caveats: []
```

Every entry follows this shape: `workload`, `request`, `observed` (the actual measured numbers, captured live), `mechanism` (plain English), and `expect` (`rules_fired`, `confidence`, `caveats`, and free-text notes flagging what a wrong result would mean). Ranges are used deliberately, not exact numbers — duty-cycled workloads vary run to run, and the file says so explicitly at the top.

6. Verification method

Every fixture's behaviour was confirmed two different ways, deliberately not relying on either alone:

1. **kubectl pod/event inspection** — pod status, container `lastState`, and scheduling events. This is what proves the *mechanism* worked (e.g. the kernel really did OOM-kill the container).
2. **Live PromQL queries against the running Prometheus**, via the API server's service proxy (no port-forward needed to stay open):

```
P=/api/v1/namespaces/monitoring/services/kps-kube-prometheus-stack-prometheus:9090/proxy/api/v1
kubectl get --raw "$P/query?query=<url-encoded PromQL>"
```

This second step matters: it is not enough that the *cluster* behaves correctly — the data has to actually reach Prometheus, with the right labels, in a form the rule engine can query. A fixture that behaves correctly in `kubectl get pods` but whose metrics never reach Prometheus (wrong namespace, missing scrape target, label mismatch) would be silently useless to Phase 4. Both checks were run for every fixture; §7 shows the actual query results.

7. Live verification results (2026-09-25, ~10 min after deploy)

Metric	Query	Result
Requests registered	<code>kube_pod_container_resource_requests{namespace="demo",resource="cpu"}</code>	All 9 pods present with correct request values (0.05–64 cores)
Restarts	<code>kube_pod_container_status_restarts_total{namespace="demo"}</code>	crashloop =5, oom =5, all others=0
Termination reasons	<code>kube_pod_container_status_last_terminated_reason{namespace="demo"} == 1</code>	crashloop only (reason present); oom only (reason present) — confirmed distinct reasons via pod inspection (Error vs OOMKilled)
Unschedulable	<code>kube_pod_status_unschedulable{namespace="demo"}</code>	pending =1, only entry
HPA object	<code>kube_horizontalpodautoscaler_info{namespace="demo"}</code>	1 series: hpa-coupled → Deployment/hpa-coupled
CPU usage rate	<code>rate(container_cpu_usage_seconds_total{namespace="demo",container="c"}[2m])</code>	right-sized 0.404 (404m); cpu-over-requested 0.058 (58m); cpu-spike 0.284 (284m, caught mid-transition); new-workload 0.00007 (~0m)
Memory working set	<code>container_memory_working_set_bytes{namespace="demo",container="c"}</code>	right-sized 71.0MB; memory-over-requested 59.9MB; hpa-coupled 5.7MB; new-workload 6.6MB; cpu-over-requested 0.6MB; cpu-spike 0.4MB

All 9 scenarios verified through real Prometheus data, not just `kubectl top` snapshots — `kubectl top` only reflects the latest metrics-server sample (no history); the numbers above come from PromQL rate/instant queries against actual scraped time series, which is the same data path the analyzer's rule engine will use.

8. Re-verification 4.5 hours later

Before closing Phase 2, everything was checked again independently — not just trusted from the first pass — to confirm the fixtures are *stable* over time, not just correct at the moment of creation.

Check	First check (~10 min)	Re-check (4.5 h later)
Node	Ready	Ready, uptime 16h, no reboot gap
crashloop restarts	5	37 (backoff widening toward the 5-min cap, as expected)
oom status	OOMKilled, 5 restarts	CrashLoopBackOff, 36 restarts, still OOMKilled
pending	Pending, Insufficient cpu	Still Pending, same reason
Prometheus data freshness	—	Last kubelet scrape 4.5s old — no gap
Active series	15,472 (end of Phase 1)	15,462 (stable, demo namespace's ~17 series balanced by some Phase-1 lab series expiring)
Prometheus PVC size	12 MB (end of Phase 1)	75 MB
GitHub	—	All 5 PRs merged; local <code>main</code> in sync; no stray branches

Nothing had drifted or gone stale. This re-check is what the answer to "is Phase 2 completely finished?" was actually based on — re-running the verification queries live rather than repeating the earlier claim from memory.

9. What Phase 2 confirms for the rule engine (R001-R005)

Fixture evidence	Design consequence for Phase 4
right-sized stays above both 50% thresholds	R001/R002's core threshold logic has a working negative control to test against — a rule engine that always fires would fail this fixture specifically
cpu-over-requested at 5.5–8.3% of request, zero throttling	A clean positive case for R001 with no confounding guard active
cpu-spike caught bursting to 999m after long idle	Concrete data to compute p99/p50 against — the bursty guard has a real fixture, not just a described one
oom vs crashloop have different termination reasons (OOMKilled vs Error), same restart pattern	Proves R004 must branch on reason, not just "did it restart" — reusing R003's restart signal alone would conflate the two
hpa-coupled looks like the strongest over-provisioning case by raw ratio, but has a real HPA object	R001 needs an explicit HPA-detection step before emitting a resize; this fixture is a trap for a rule that skips that check
new-workload is deployed with the others but has R005-relevant "youth" regardless of window length	R005 must gate on data window length against each rule's own minimum, not on wall-clock creation time
pending has zero usage data (never scheduled)	Confirms the dashboard/rule engine must handle a workload with no metrics at all, not just sparse ones

10. Resource footprint

Item	Value	Label
New image pulls	0 (both images already cached from Phase 1)	MEASURED
Node container memory	2.32-2.64 GiB (up from ~2.2 GiB at end of Phase 1)	MEASURED
Host available memory	7.2-8.0 GiB throughout	MEASURED
Peak combined demo CPU	< 1.5 cores (only #2's duty cycle and #4's burst can briefly overlap); steady-state well under 1 core	ESTIMATED
Prometheus active series	15,462 (stable; demo added ~17, offset by Phase-1 lab series expiring)	MEASURED
Prometheus PVC	75 MB, well under the 5GB retention cap	MEASURED

11. Git and GitHub workflow

Commit	Content
1c56dbb	Merge of PR #4 — 6 ADRs (D-001,002,003,005,006,007) with Phase 1 evidence
310f567	Phase 2: 9 demo workloads, ground truth, doc updates
6570169	Merge of PR #5 — current main

Same conventions as Phase 1: work on a branch (`phase-2`), stage files by name, scan for secrets before staging, Conventional-Commit messages, merge via a merge commit, delete the branch both locally and on GitHub once merged.

```
git switch -c phase-2
grep -riE 'password|secret|token|BEGIN .*KEY' demo/ # scan before staging
git add demo/ PHASES.md docs/requirements.md
git commit -m "feat(phase-2): demo workloads, verified against live Prometheus"
git push -u origin phase-2
gh pr create --base main --head phase-2 --title "..." --body "..."
gh api -X PUT repos/ISHANT57/kubernetes-workload-analyzer/pulls/5/merge -f merge_method=merge
git switch main && git pull --ff-only
git branch -d phase-2 && git push origin --delete phase-2
```

12. Current state, operating and rollback

12.1 State at 2026-09-25 13:53 MEASURED

Namespace	Running
demo	9 Deployments (7 Running, crashloop and oom cycling in CrashLoopBackOff by design, pending Unscheduleable by design), 1 Service, 1 HPA

12.2 Operating

```
kubectl config use-context kind-workload-analyzer
kubectl -n demo get pods -o wide
kubectl -n demo get hpa hpa-coupled
P=/api/v1/namespaces/monitoring/services/kps-kube-prometheus-stack-prometheus:9090/proxy/api/v1
kubectl get --raw "$P/query?query=kube_pod_container_resource_requests%7Bnamespace%3D%22demo%22%7D"
```

12.3 Rollback / removal

```
kubectl delete -k demo/ # removes all 9 workloads, the Service and the HPA
kubectl delete namespace demo # equivalent, coarser
```

13. Open items and next phase

Item	Status
D-004 (no application database) ADR	Still PROPOSED — deferred until accepted
Cluster recovery after Docker restart / reboot	Still deferred to Phase 7 (untested for 16h+ of continuous uptime so far)
Series count re-check	Worth a periodic glance as more phases add scrape targets, but 15.5k against the stack's practical headroom leaves ample margin
Phase 3 — backend skeleton	Next: Go module — config, Prometheus client, Kubernetes client (D-007 ClusterRole scope), analysis-run object, <code>/healthz//readyz//metrics</code> , structured logs. No rule logic yet.

Phase 2 leaves Phase 3 with a cluster that already has real, varied, labelled Prometheus data to query from the very first line of backend code — no synthetic fixtures will be needed to exercise the Prometheus client.

Appendix A — Manifest reference

File	Object(s)	Lines
demo/00-namespace.yaml	Namespace demo	5
demo/01-right-sized.yaml	Deployment	23
demo/02-cpu-over-requested.yaml	Deployment	19
demo/03-memory-over-requested.yaml	Deployment	18
demo/04-cpu-spike.yaml	Deployment	20
demo/05-oom.yaml	Deployment	19
demo/06-crashloop.yaml	Deployment	17
demo/07-hpa-coupled.yaml	Deployment + Service + HPA	38
demo/08-new-workload.yaml	Deployment	19
demo/09-pending.yaml	Deployment	17
demo/expected-findings.yaml	Ground truth for all 9 scenarios	113
demo/kustomization.yaml	Applies all of the above as one unit	13

All Deployments carry `replicas: 1` and the label `k8sa.dev/scenario=<name>` on both the Deployment and its pod template.

Appendix B — Verification query reference

Question	Query
Are requests registered?	<code>kube_pod_container_resource_requests{namespace="demo",resource="cpu"}</code>
Restart counts	<code>kube_pod_container_status_restarts_total{namespace="demo"}</code>
Why did a container last stop?	<code>kube_pod_container_status_last_terminated_reason{namespace="demo"} == 1</code>
Is a pod unschedulable?	<code>kube_pod_status_unschedulable{namespace="demo"}</code>
Does an HPA exist for this workload?	<code>kube_horizontalpodautoscaler_info{namespace="demo"}</code>
CPU usage rate	<code>rate(container_cpu_usage_seconds_total{namespace="demo",container="c"}[2m])</code>
Current memory working set	<code>container_memory_working_set_bytes{namespace="demo",container="c"}</code>
Query via API-server proxy (no lingering port-forward)	<code>kubectl get --raw "/api/v1/namespaces/monitoring/services/kps-kube-prometheus-stack-prometheus:9090/proxy/api/v1/query?query=<url-encoded>"</code>
Pod-level termination detail	<code>kubectl -n demo get pod <p> -o jsonpath='{.status.containerStatuses[0].lastState}'</code>
Scheduling failure reason	<code>kubectl -n demo get events --field-selector reason=FailedScheduling</code>
Live usage snapshot (no history)	<code>kubectl -n demo top pods</code>